程序设计原理

递归

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

Objectives

- To know what is a recursive function and the benefits of using recursive functions .
- To determine the base cases in a recursive function .
- To understand how recursive function calls are handled in a call stack .
- To understand the relationship and difference between recursion and iteration .
- To solve problems using recursion .

What is Recursion?

- **Recursion** means a function calls itself.
- A function solves a big problem by repeatedly solving smaller ones.

factorial(0) = 1;
factorial(n) = n*factorial(n-1);

Why Do We Need Recursion?

- ✓ Recursion expresses complex problems in a simple and natural way.
- ✓ Ideal for trees, divide-and-conquer, and mathematical definitions.（遍历文件夹）

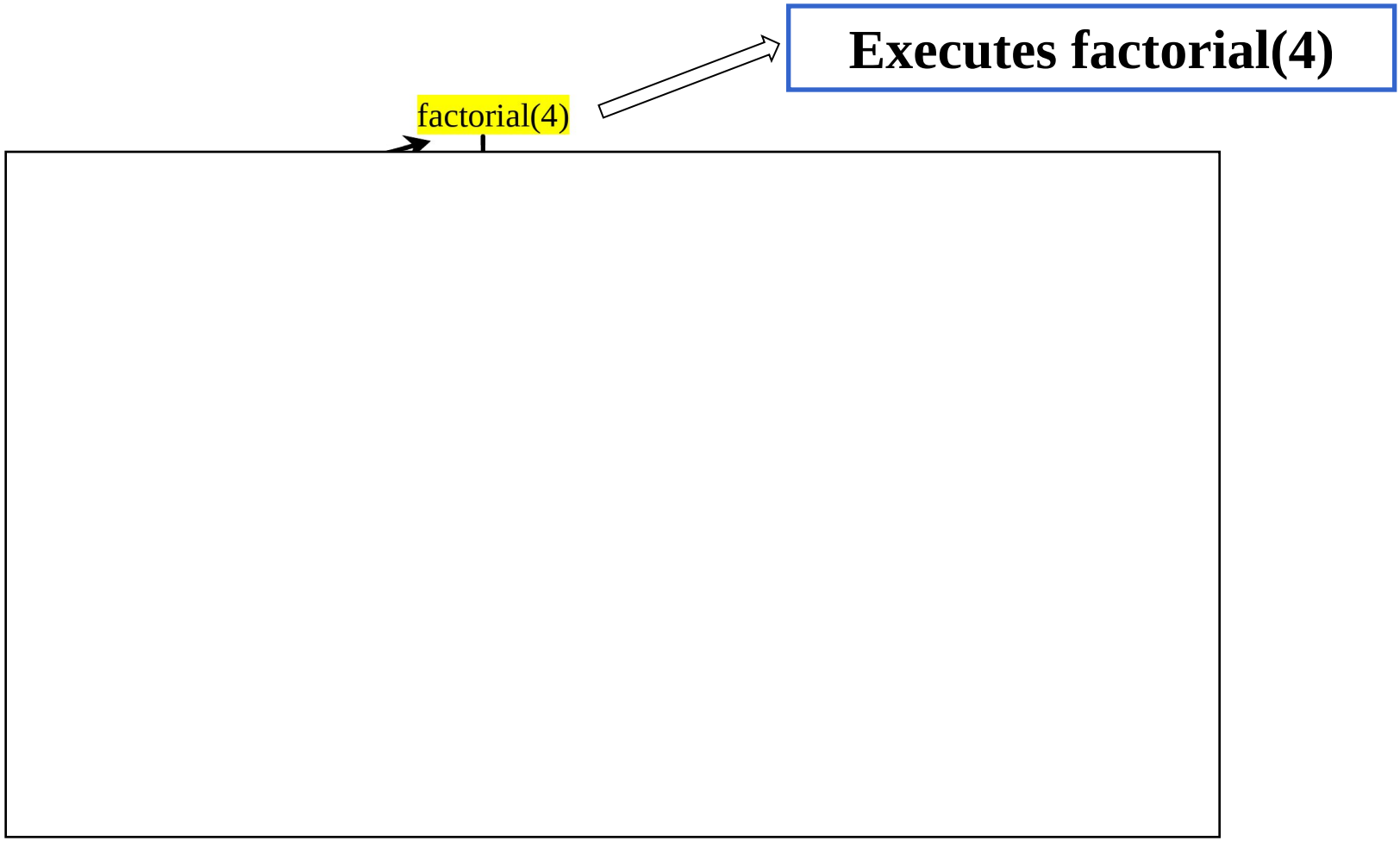
Computing Factorial

factorial(0) = 1;

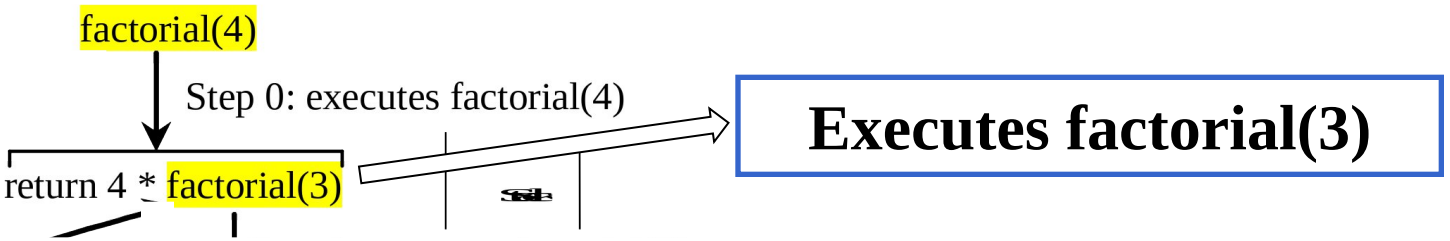
factorial(n) = n*factorial(n-1);

```
1  // Return the factorial for a specified index
2  int factorial(int);
3  int main(){
4    // Prompt the user to enter an integer
5    cout << "Please enter a non-negative integer: ";
6    int n;
7    cin >> n;
8    // Display factorial
9    cout << "Factorial of " << n << " is " << factorial(n);
10   return 0;
11 }
12 // Return the factorial for a specified index
13 int factorial(int n){
14   if (n == 0) // Base case
15     return 1;
16   else
17     return n * factorial(n - 1); // Recursive call
18 }
```

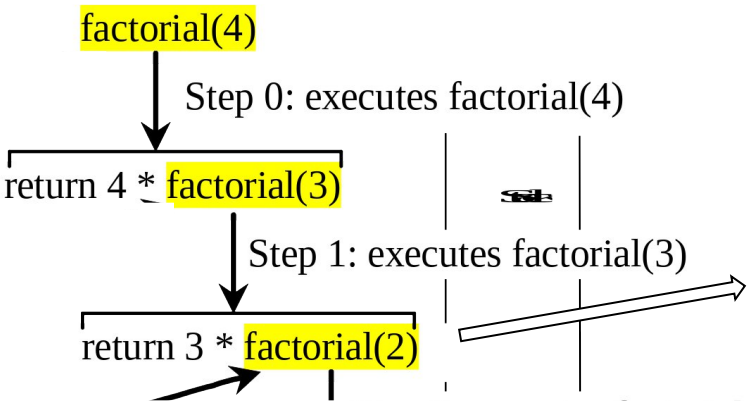
Trace Recursive factorial



Trace Recursive factorial

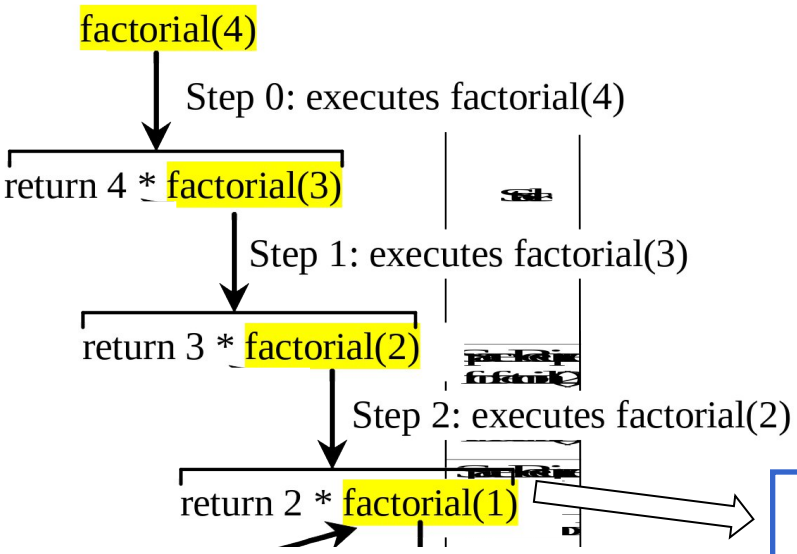


Trace Recursive factorial



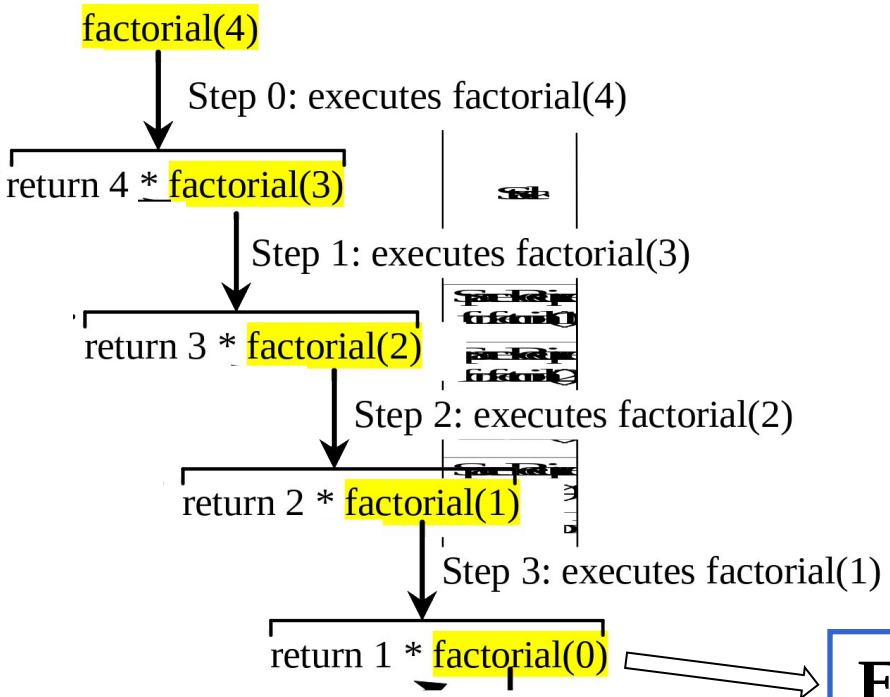
Executes factorial(2)

Trace Recursive factorial

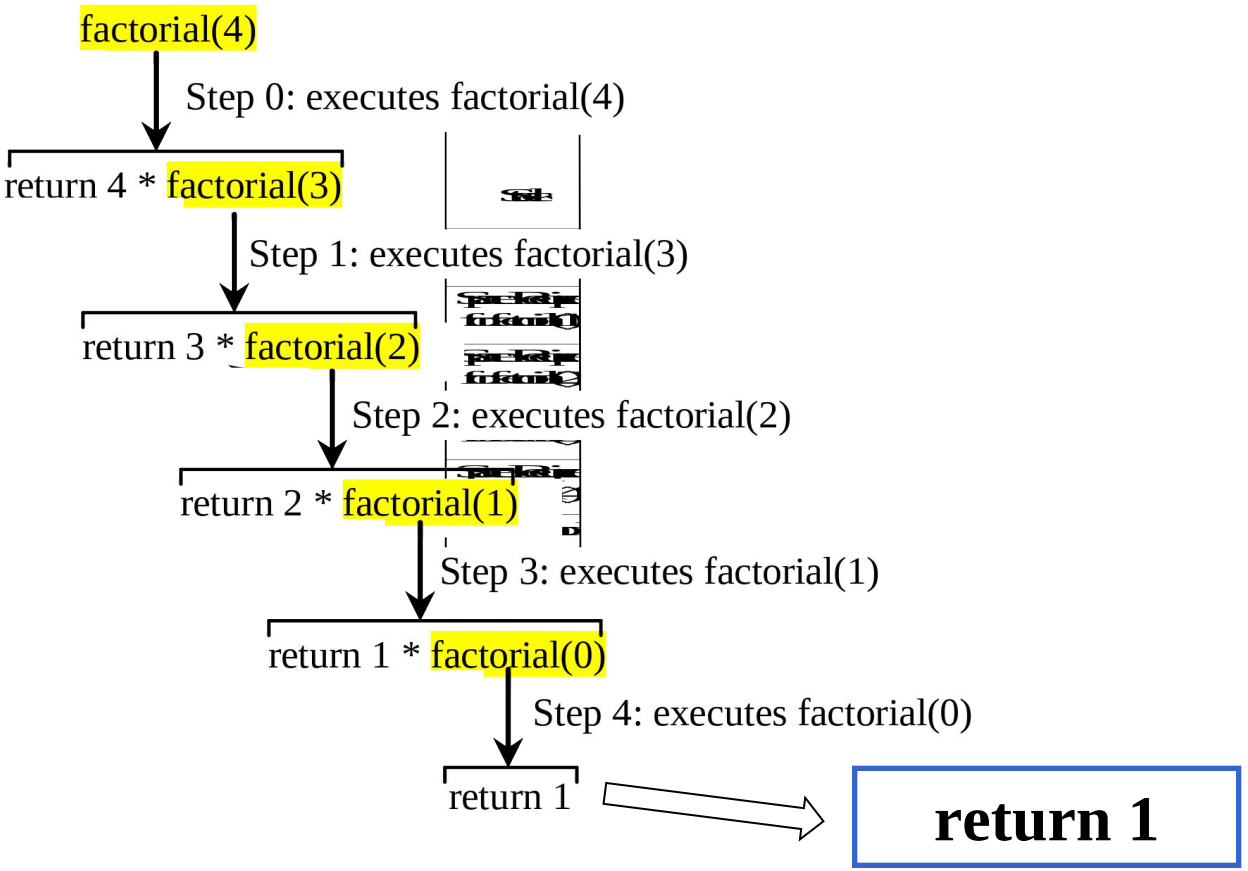


Executes factorial(1)

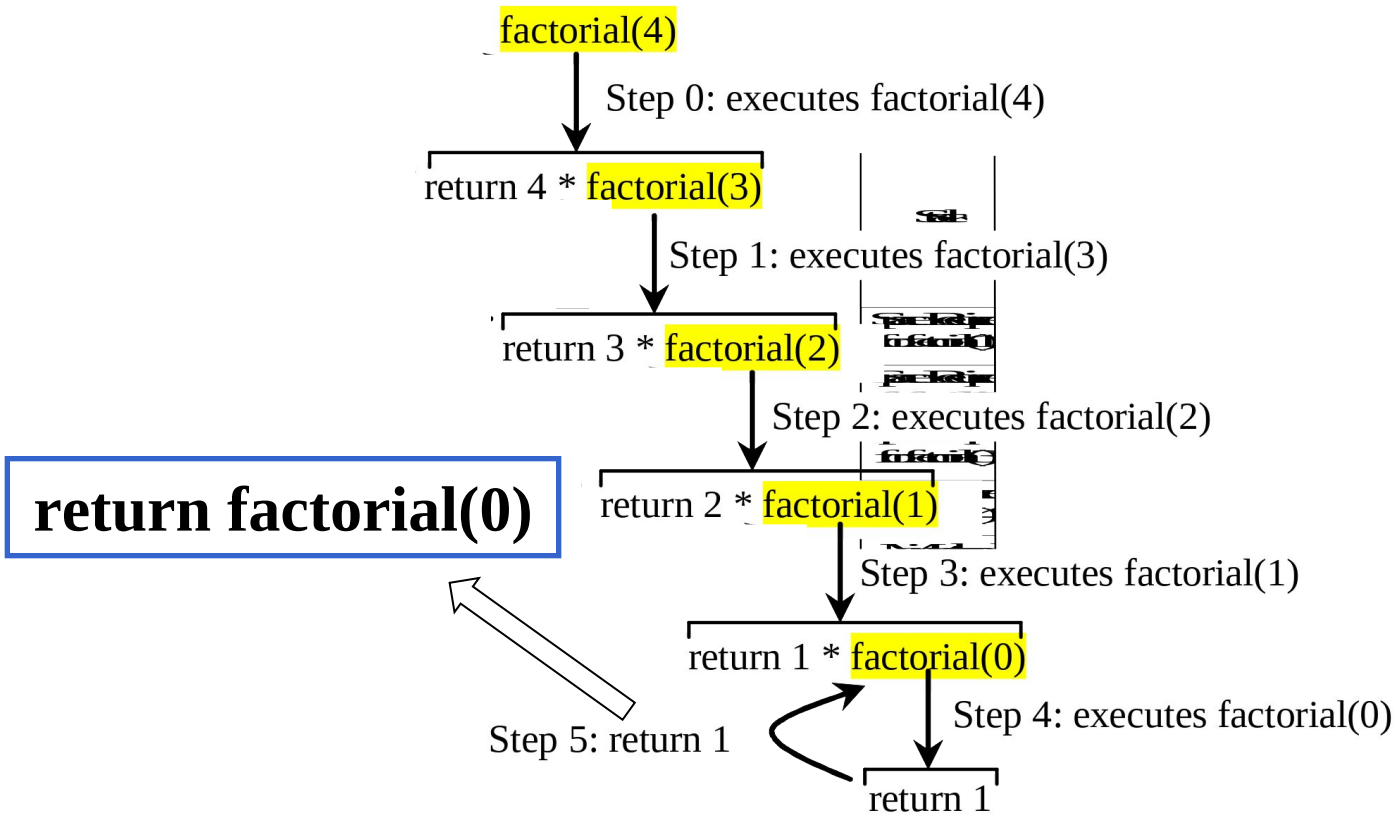
Trace Recursive factorial



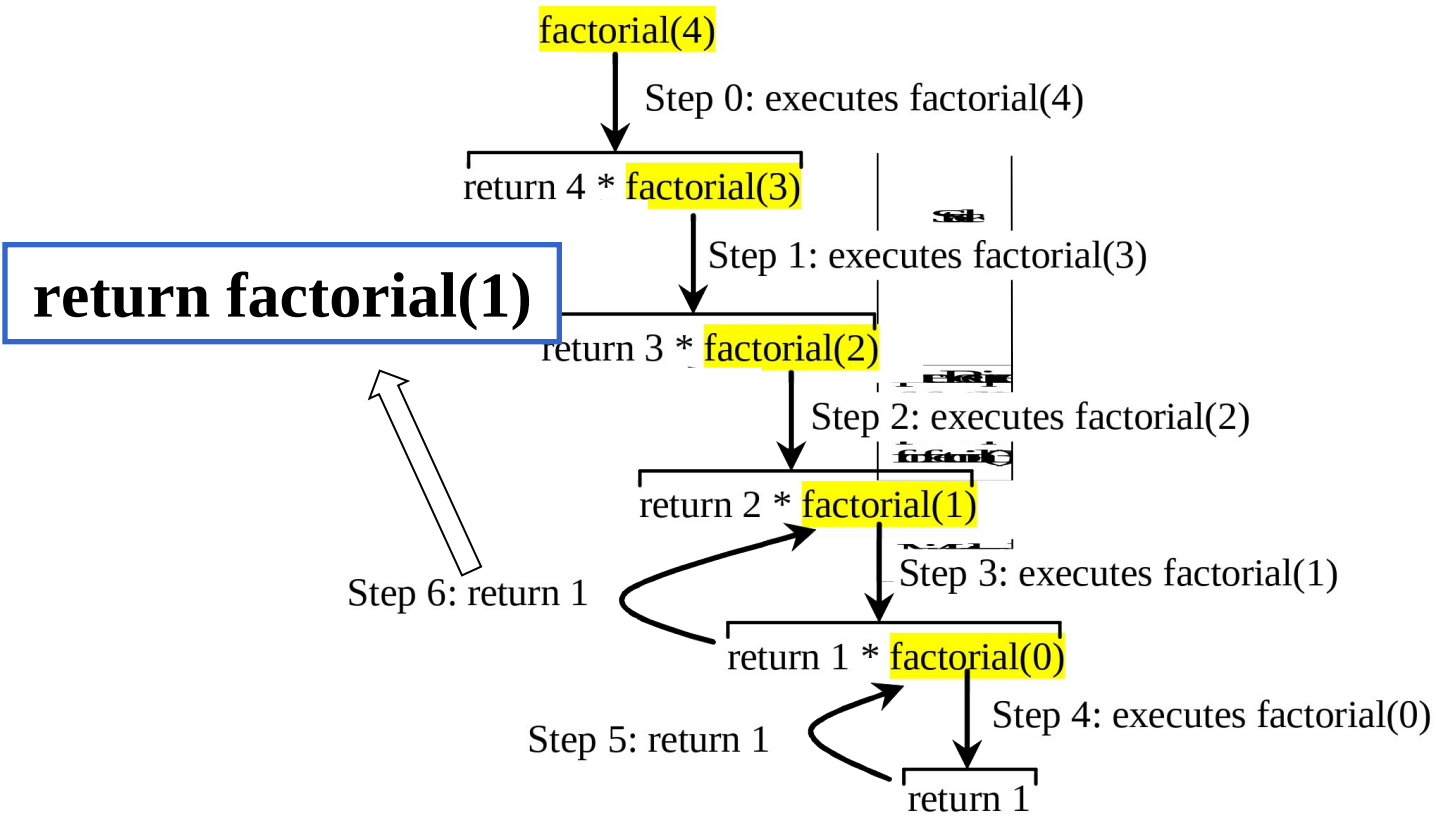
Trace Recursive factorial



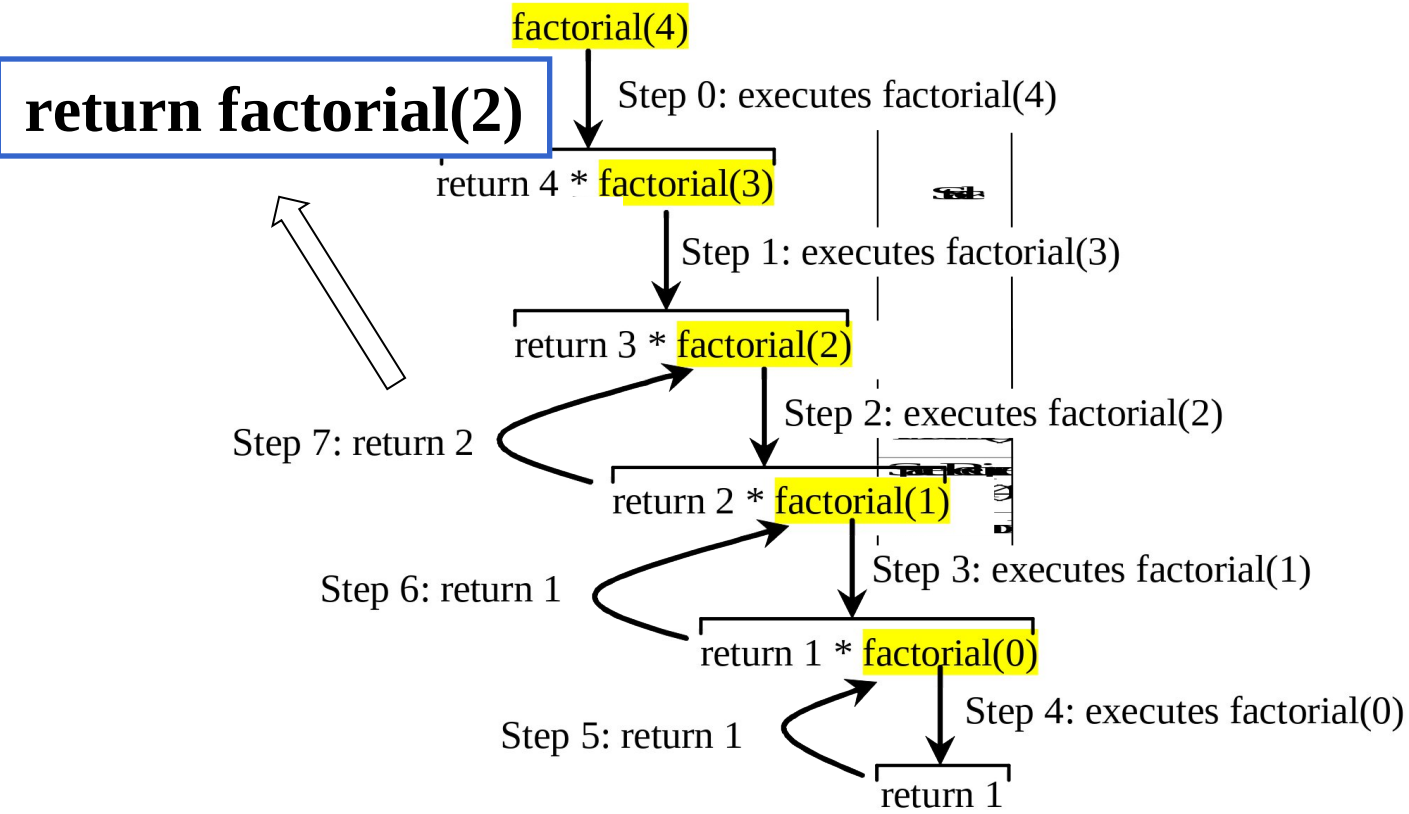
Trace Recursive factorial



Trace Recursive factorial

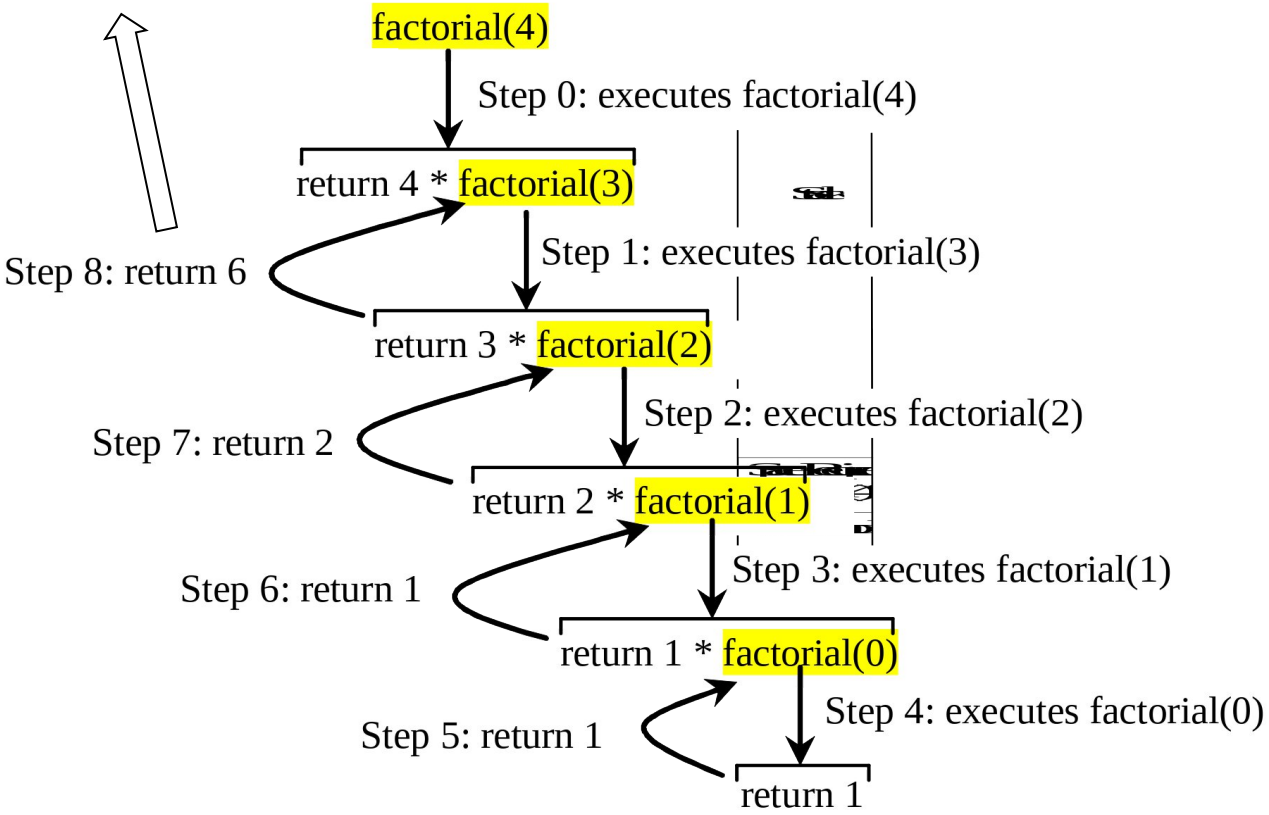


Trace Recursive factorial



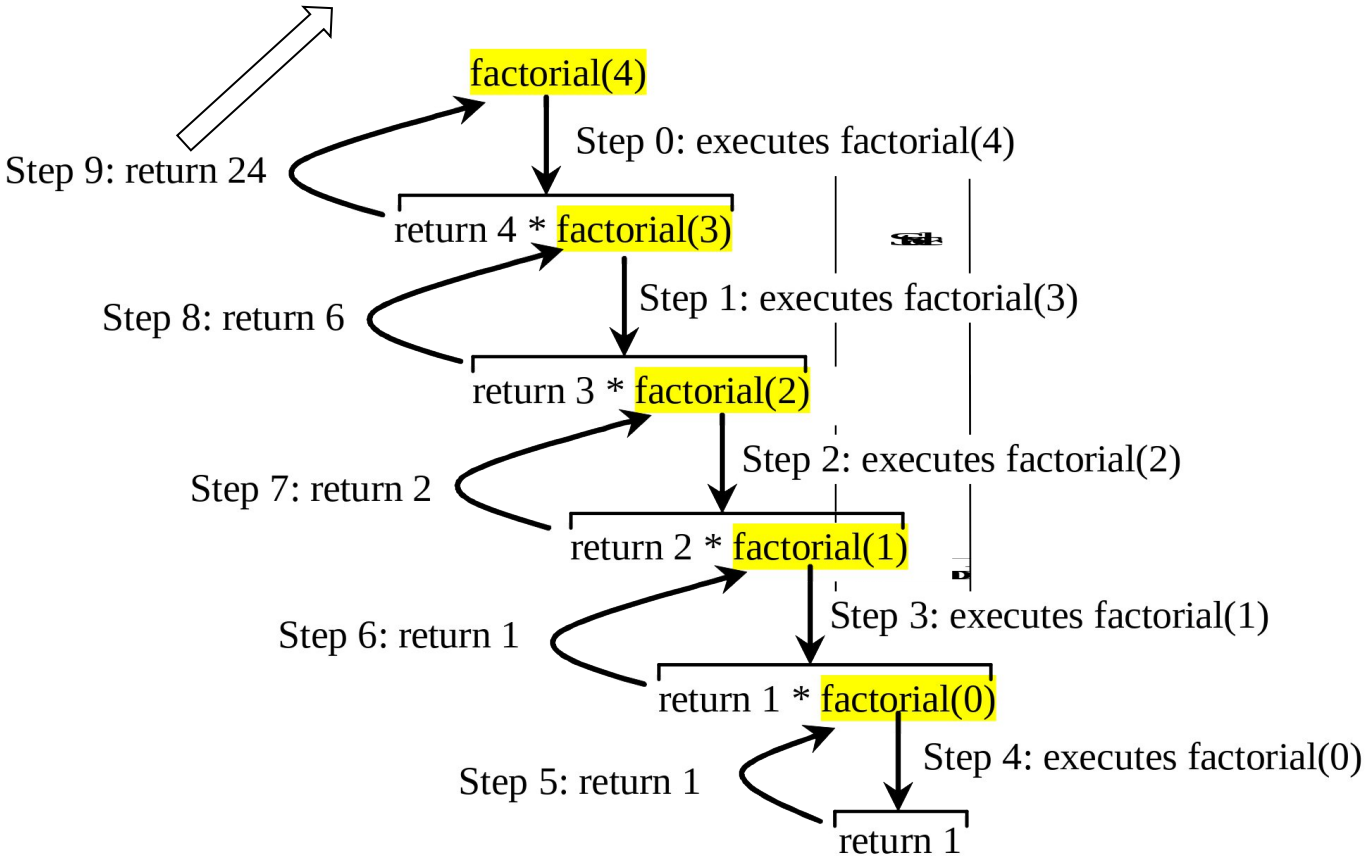
Trace Recursive factorial

return factorial(3)

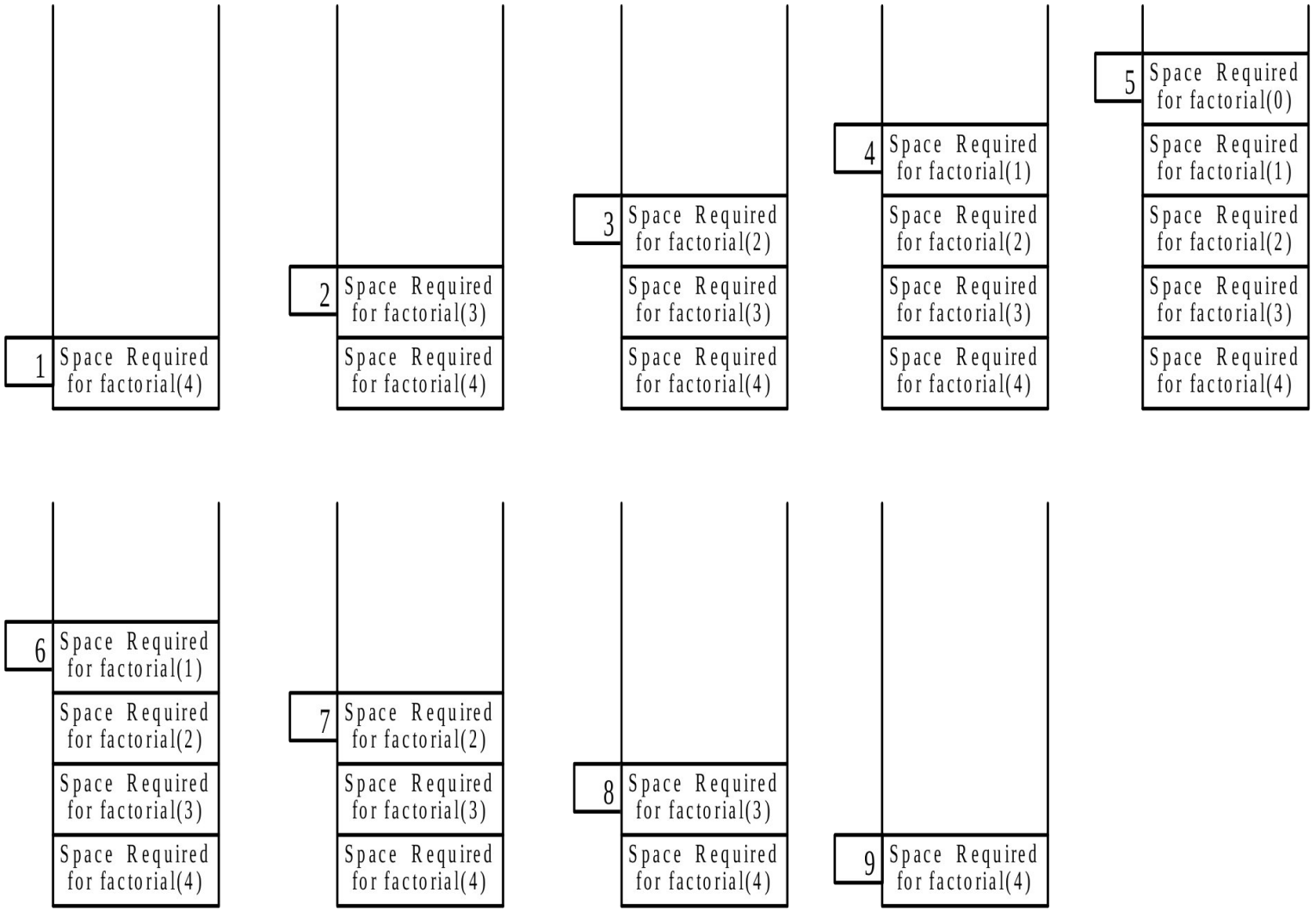


Trace Recursive factorial

return factorial(4)



factorial(4) Stack Trace



Fibonacci Numbers

Finonacci series: 0 1 1 2 3 5 8 13 21 34 55 89…
indices: 0 1 2 3 4 5 6 7 8 9 10 11

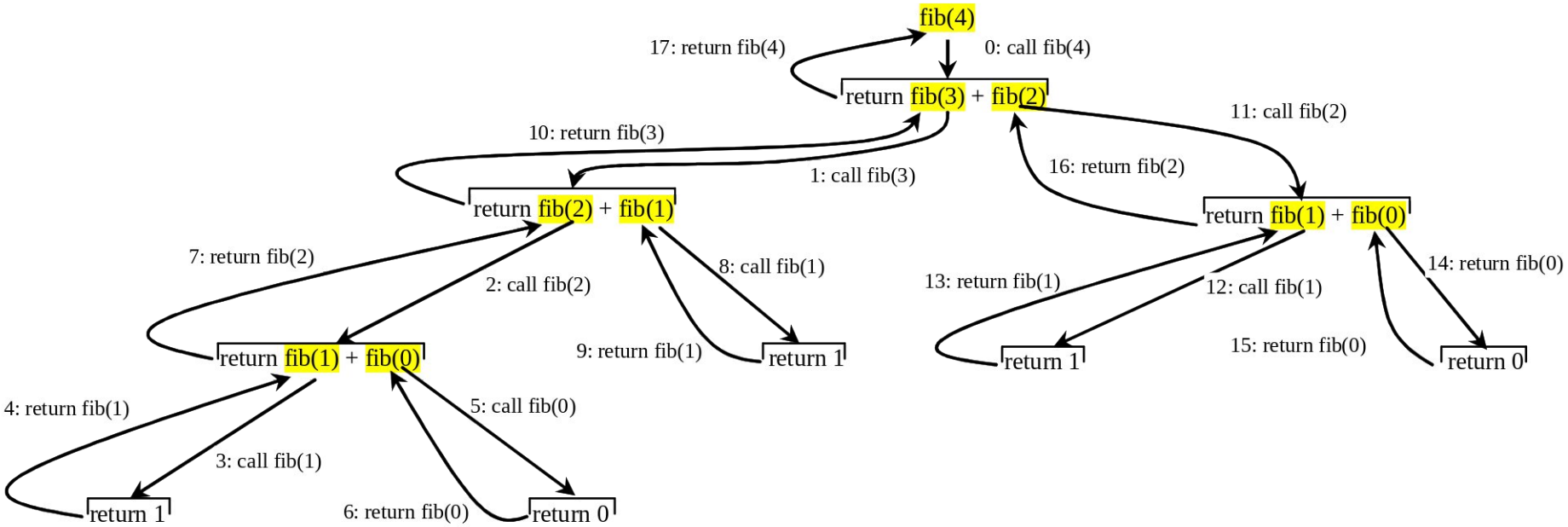
fib(0) = 0;
fib(1) = 1;
fib(index) = fib(index -1) + fib(index -2); index >=2

fib(3) = fib(2) + fib(1) = (fib(1) + fib(0)) + fib(1) = (1 + 0) +fib(1) = 1 + fib(1) = 1 + 1 = 2

Fibonacci Numbers Function

```
1  // The function for finding the Fibonacci number
2  int fib(int);
3  int main(){
4    // Prompt the user to enter an integer
5    cout <<  "Enter an index for the Fibonacci number: ";
6    int index;
7    cin >> index;
8    // Display factorial
9    cout << "Fibonacci number at index " << index << " is "
10     << fib(index) << endl;
11   return 0;
12 }
13 // The function for finding the Fibonacci number
14 int fib(int index){
15   if (index == 0) // Base case
16     return 0;
17   else if (index == 1) // Base case
18     return 1;
19   else // Reduction and recursive calls
20     return fib(index - 1) + fib(index - 2);
21 }
```

Fibonnaci Numbers, cont.



Exercise

When invoke f(10), how many times will function f() be called?

```
int f(int x){
    if (x <= 2) //Base case
        return 1;
    else
        return f(x-2) + f(x-4) + 1;
}
```

A. 10
B. 15
C. 16
D. 20

Characteristics of Recursion

All recursive methods have the following characteristics:

- One or more base cases (the simplest case) are used to stop recursion.
- Every recursive call reduces the original problem, bringing it increasingly closer to a base case until it becomes that case.

In general, to solve a problem using recursion, you break it into subproblems. If a subproblem resembles the original problem, you can apply the same approach to solve the subproblem recursively. This subproblem is almost the same as the original problem in nature with a smaller size.

Recursion and Iteration

In the previous sections, we studied two recursive functions that can also be implemented with simple iterative programs. This section compares the two approaches and discusses why you might choose one approach over the other in a particular situation.

Recursion and Iteration

- **Factorial Implementation**

```
1  // iterative function factorial
2  unsigned long factorial(unsigned int number){
3      unsigned long result = 1;
4
5      // iterative factorial calculation
6      for(unsigned int i = number; i >= 1; --i)
7          result *= i;
8
9      return result;
10 }// end function factorial
```

```
1  // recursive definition of function factorial
2  unsigned long factorial(unsigned long number){
3      if(number <= 1) //test for base case
4          return 1;//base cases:0! = 1 and 1! = 1
5      else // recursion step
6          return number * factorial(number - 1);
7  }// end function factorial
```

Recursion and Iteration

- Both iteration and recursion are based on a control statement: Iteration uses a repetition structure; recursion uses a selection structure.
- Both iteration and recursion involve repetition: Iteration explicitly uses a repetition structure; recursion achieves repetition through repeated function calls.
- Iteration and recursion each involve a termination test: Iteration terminates when the loop-continuation condition fails; recursion terminates when a base case is recognized.
- Iteration with counter-controlled repetition and recursion each gradually approach termination: Iteration modifies a counter until the counter assumes a value that makes the loop-continuation condition fail; recursion produces simpler versions of the original problem until the base case is reached.
- Both iteration and recursion can occur infinitely: An infinite loop occurs with iteration if the loop-continuation test never becomes false; infinite recursion occurs if the recursion step does not reduce the problem during each recursive call in a manner that converges on the base case.

Recursive Binary Search

Case 1: If the key is less than the middle element, recursively search the key in the first half of the array.

Case 2: If the key is equal to the middle element, the search ends with a match.

Case 3: If the key is greater than the middle element, recursively search the key in the second half of the array.

Recursive Implementation

```
1  int binarySearch(const int list[], int key, int low, int high)
2  {
3    if (low > high)  // The list has been exhausted without a match
4      return -low - 1; // Return -insertion point - 1
5    int mid = (low + high) / 2;
6    if (key < list[mid])
7      return binarySearch(list, key, low, mid - 1);
8    else if (key == list[mid])
9      return mid;
10   else
11     return binarySearch(list, key, mid + 1, high);
12 }
13 int binarySearch(const int list[], int key, int size){
14   int low = 0;
15   int high = size - 1;
16   return binarySearch(list, key, low, high);
17 }
18 int main(){
19   int list[] = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};
20   int i = binarySearch(list, 2, 13); // returns 0
21   int j = binarySearch(list, 11, 13); // returns 4
22   int k = binarySearch(list, 12, 13); // returns -6
23   cout << "binarySearch(list, 2, 13) returns " << i << endl;
24   cout << "binarySearch(list, 11, 13) returns " << j << endl;
25   cout << "binarySearch(list, 12, 13) returns " << k << endl;
26   return 0;
27 }
```

Towers of Hanoi

- There are *n* disks labeled 1, 2, 3, . . ., *n*, and three towers labeled A, B, and C.
- No disk can be on top of a smaller disk at any time.
- All the disks are initially placed on tower A.
- Only one disk can be moved at a time, and it must be the top disk on the tower.

Towers of Hanoi, cont.

A B C

Step 0: Starting status

A B C

Step 1: Move disk 1 from A to B

A B C

Step 2: Move disk 2 from A to C

A B C

Step 3: Move disk 1 from B to C

A B C

Step 4: Move disk 3 from A to B

A B C

Step 5: Move disk 1 from C to A

A B C

Step 6: Move disk 2 from C to B

A B C

Step 7: Mve disk 1 from A to B

Solution to Towers of Hanoi

```
1  /* The function for finding the solution to move n disks
2       from fromTower to toTower with auxTower */
3  void moveDisks(int n, char fromTower,
4       char toTower, char auxTower){
5    if (n == 1) // Stopping condition
6      cout << "Move disk " << n << " from " <<
7        fromTower << " to " << toTower << endl;
8    else{
9      moveDisks(n - 1, fromTower, auxTower, toTower);
10     cout << "Move disk " << n << " from " <<
11       fromTower << " to " << toTower << endl;
12     moveDisks(n - 1, auxTower, toTower, fromTower);
13   }
14 }
15 int main(){
16   // Read number of disks, n
17   cout << "Enter number of disks: ";
18   int n;
19   cin >> n;
20   // Find the solution recursively
21   cout << "The moves are: " << endl;
22   moveDisks(n, 'A', 'B', 'C');
23   return 0;
24 }
```

Exercise

Fill in the code to complete the following function for checking whether a string is Palindrome.

```
bool isPalindrome(const char * const s, int low, int high){
        if (high <= low) //Base case
                return true;
        else if (s[low] != s[high]) //Base case
                return false;
        else return _________;
}
```

A. isPalindrome(s, low, high);
B. isPalindrome(s, low+1, high);
C. isPalindrome(s, low, high-1);
D. isPalindrome(s, low+1, high-1);

Exercise

Description

Find the greatest common divisor of two numbers.

You are asked to use recursive definition.

Input

The input data contains multiple test instances. Each set of data takes up a row with two positive integers **n** and **m** in each row. The input ends with **0 0**.

Output

For each set of input samples, output a number that is their greatest common divisor.

Sample input

15 5
20 3
65 13
0 0

Sample output

5
1
13

Hint

$1<=n,m<=10^{15}$